

INFORMATION RETRIEVAL ASSIGNMENT REPORT

1. Objective

The objective of this project is to design and implement an end-to-end Information Retrieval system using Streamlit. The application allows users to upload a document collection, preprocess text, build indexes, execute queries, and perform tolerant retrieval.

2. Dataset

Dataset Name: Wikipedia Movie Plots Dataset

Source: Kaggle

Number of Documents Used: 1000 Movie Plot Documents

Columns Used:

- Title
- Genre
- Plot

The Plot column is treated as the document collection for retrieval.

3. Streamlit Workflow

The Streamlit application provides the following functionalities:

1. Upload Dataset
2. Display Documents
3. Enter Query
4. Select Retrieval Technique
5. Display Intermediate Results
6. Display Final Search Results

Screenshot 1

(Insert Dataset Upload Screenshot)

Screenshot 2

(Insert Query Processing Screenshot)

4. Text Preprocessing

The following preprocessing techniques were applied:

Tokenization

Text was split into individual words.

Lowercasing

All words were converted to lowercase.

Stopword Removal

Common words such as "the", "is", and "and" were removed.

Hyphen Handling

Hyphenated words were separated.

Stemming

Words were reduced to root forms using Porter Stemmer.

Lemmatization

Words were converted into meaningful dictionary forms.

Screenshot 3

(Insert Preprocessing Output Screenshot)

5. Inverted Index

An inverted index was created to map terms to the documents in which they occur.

Example:

hero → [1, 5, 12]

villain → [3, 10, 25]

love → [4, 8, 15]

Screenshot 4

(Insert Inverted Index Screenshot)

6. Stemming vs Lemmatization

Example Comparison

Word	Stemmed	Lemmatized
running	run	run
studies	studi	study
heroes	hero	hero

Observation

Lemmatization preserved semantic meaning better than stemming.

Conclusion

Lemmatization is more suitable for the Wikipedia Movie Plots Dataset.

7. Phrase Query Processing

Query Example:

love story

Biword Index

(love story) $\rightarrow$ [14, 27, 50]

Positional Index

love: Doc14:[10]

story: Doc14:[11]

Observation

Biword Index may generate false positives.

Positional Index verifies exact positions and provides more accurate results.

Conclusion

Positional Index is more accurate.

Screenshot 5

(Insert Phrase Query Screenshot)

8. BST vs B-Tree Comparison

Experimental Results

Query	BST(ms)	B-Tree(ms)
love	0.32	0.08
hero	0.29	0.07
villain	0.31	0.09
mission	0.28	0.06

Observation

B-Tree consistently required less search time.

Conclusion

B-Tree performed better than BST.

Screenshot 6

(Insert BST/B-Tree Screenshot)

9. Tolerant Retrieval

Misspelled Query	Corrected Query
spce misson	space mission
herro	hero
seriel killer	serial killer

Observation

Edit Distance successfully corrected spelling errors and retrieved relevant documents.

Screenshot 7

(Insert Tolerant Retrieval Screenshot)

10. Inference and Discussion

Which preprocessing technique improved retrieval quality?

Lowercasing, stopword removal, and lemmatization improved retrieval quality.

Was stemming or lemmatization better?

Lemmatization performed better.

Which phrase query index was more accurate?

Positional Index.

Which tree structure was faster?

B-Tree.

How tolerant was the retrieval model?

The model successfully corrected misspelled queries and returned relevant documents.

Limitations

- Exact keyword dependency
- No semantic search
- No ranking model

Future Improvements

- TF-IDF
 - BM25
 - Word2Vec
 - BERT-based Retrieval
 - Phonetic Correction
-

11. Conclusion

The project successfully implemented an end-to-end Information Retrieval system using Streamlit. The system demonstrated preprocessing, indexing, phrase query processing, dictionary search, and tolerant retrieval on the Wikipedia Movie Plots Dataset.